

Final Report

Math 748

Nicholas Kane

May 22, 2026

1 Introduction

Although not the media titan it was mid 20th century, the film industry still holds a major influence on entertainment culture. There are many factors that quantify a films success, the main two being quality and popularity. This project seeks to use data from IMDb to study how various qualities of a film influence its rating on the site. Although there are numerous film scoring sites, IMDb offers their data for use in projects, and the data is fairly clean. Average rating on a database may not be the general feeling towards a film, but it is one way of quantifying a specific populations feeling about them. Linear regression, ridge/lasso, support vector machines, and random forests will be used to investigate and quantify this relationship. The main goal is to figure out which factors influence average rating, and to find an adequate model for prediction.

2 Dataset and Features

The data set used in this project is combined from two datasets on IMDb (Internet Movie Database). The two datasets are `title.basics`, which contain metadata about the movies such as title and genre, and the second is `title.ratings`, which contain both the rating score and the number of ratings. This data set is updated everyday, however I am using the data set downloaded on February 3, 2026. This should not impact the model much, as relatively few films will be released in this window.

The `title.basics` data set contains 12,263,801 rows which consist of movies, TV shows, shorts, and a few more visual media types. The focus of the project is on feature length films, which is the only type of interest.

The `title.ratings` data set contains 1,631,482 rows which consist of the ratings for all the media on the site. It is a smaller number since not everything on the site is rated, and some TV shows get rated by season rather than by episode. That discrepancy should not matter, since I will only be looking at movies that have a rating.

Source: Dataset

2.1 Data Augmentation

A fair bit of pre-processing was necessary before implementing any statistical learning methods. The first step was to remove any variables not important to the model. These variables were `titleType` $\neq$ `movie`, `isAdult`, `originalTitle`, and `endYear`. After these variables were removed, I could merge the tables on `tconst` as seen in figure 1.

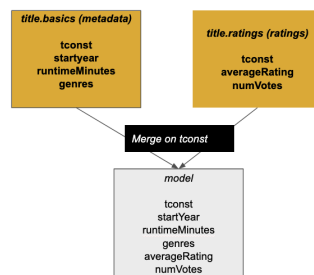


Figure 1: Data Pipeline

Next, the missingness had to be analyzed. The table below shows the missingness for the remaining variables. It is evident that `primaryTitle` and `startYear` can be removed with no consequence, though `runtimeMinutes` and `genres` required additional analysis.

Variable	Missing values	Percent MV
tconst	0	0.00
primaryTitle	2	0.00
startYear	35	0.01
runtimeMinutes	32817	9.83
genres	11051	3.31
averageRating	0	0.00
numVotes	0	0.00

Table 1: Missing Values

Figures 2 and 3 show that there are large peaks of missing information before the 1920s and 1930s. Afterwards, the missing information stabilizes. Those decades saw movies in their infancy and many pictures from that era are now lost, meaning data about them is also lost. The red line is 10%, or a proportion of 0.1, placed for ease of viewing when the missingness mellows out. Due to these large counts in early years, as well as changes in the film industry from the classical Hollywood system to the modern system, I will eliminate all years before 1970.

Once the years were filtered, some missing information still remained, so I filtered out all columns with no runtime or genre information.

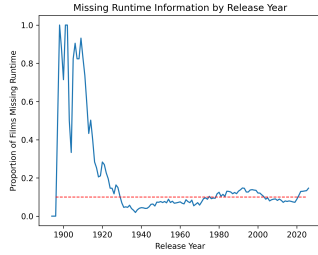


Figure 2: Missing Runtime by Year

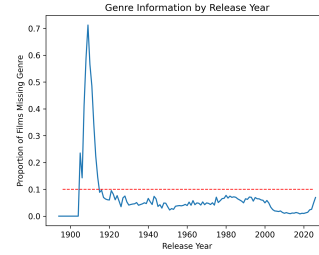


Figure 3: Missing Genres by Year

Once all missing genres are removed, the remaining column must be split into individual genre columns using one-hot encoding. The reasoning behind this particular encoding is that there are many genres, and many films contain multiple genres. One-hot encoding should allow for Since drama is the most prevalent genre, and sometimes can be considered the default genre for films, it will be removed when performing regression.

Since there is a wide gulf in the popularity of a film on IMDb, here measured by **numVotes**, I will take the log of **numVotes**. This may still have a skewed distribution, but should bring the values closer together and allow for cleaner analysis.

To deal with near zero variance, I decided to look at the remaining genres and check the proportions. There are five genres under 1%: Western, News, Reality-TV, Talk-Show, and Game-Show. Since I have restricted the data to 1970s onward, a low percentage of Westerns makes sense, and the others are TV genres that may have been misclassified as movies.

Finally, when attempting to visualize the runtime, I noticed that the histogram was difficult to read. Upon further investigation, I determined that there were some very short films and some TV miniseries that had made their way into the 'movie' type. To counteract this, I removed all movies with runtime under 60 minutes (sometimes considered the minimum length for a feature), and those over 300 minutes. Although there are a handful of true movies over 300 minutes, they are mostly specific art films, and should not affect the overall picture.

2.2 Data Visualization

As there are only four features (**startYear**, **runtimeMinutes**, **logVotes**, **genre**), I plotted each feature to understand its individual distribution.

Figure 4 shows the distribution of release years. We can see there is an uptick in movies in the 21st century. This makes sense as consumer access to digital filmmaking technologies allowed almost anyone to make and release a feature film. We can also observe a dip in 2020, which makes sense since films were shelved due to the pandemic. It shot quickly back up, though 2025 also saw a dip. This makes sense as there are still releases from the year that are being added.

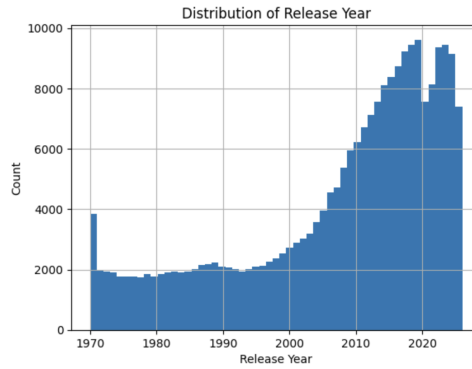


Figure 4: Distribution of Release Year

Film runtime has a right skewed distribution, and we can observe a peak around 90 minutes. This makes sense for the data since most movies do not break the 150 minute mark or two and a half hour mark besides a large scale epic or a long art film. This does not require a transformation, but will be further discussed once I get to the modeling stage. I may need to create bins for film length (short, medium, long) and use regression on those.

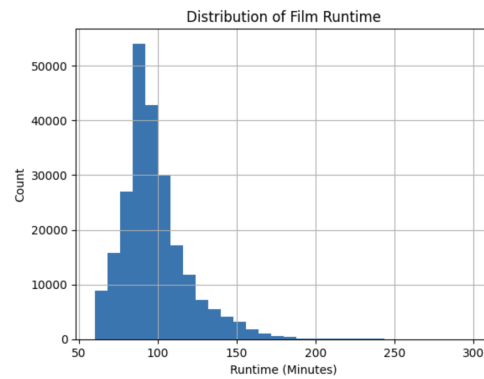


Figure 5: Distribution of Runtimes

The genre counts distribution is led by **Drama** and followed by **Comedy** and **Documentary**. Since I removed the sub 1% genres, all genres here will be a part of the initial model. However, as I enter the modeling stage, I may remove some of the smaller ones. Additionally, when modeling with Linear and Lasso/Ridge, I will remove the drama genre as that will be a baseline and I will measure

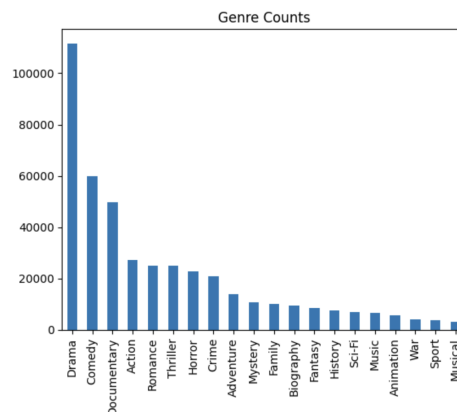


Figure 6: Distribution of Genres

When attempting to plot the distribution of raw `numVotes`, it was extremely skewed. This makes sense as there are some site favorites, and some incredibly popular movies. This can be contrasted with independent release that may have a small, but passionate following. To counteract this, I took the log of the `numVotes` column and created a new variable called `logVotes`. This still has a right skewed distribution, but a much easier

to use shape. This may contain some nonlinearity that I will explore further in the modeling stage. `logVotes` will not be included in the initial linear/lasso model, but I will model with that as a predictor to see if there is any difference in interpretation.

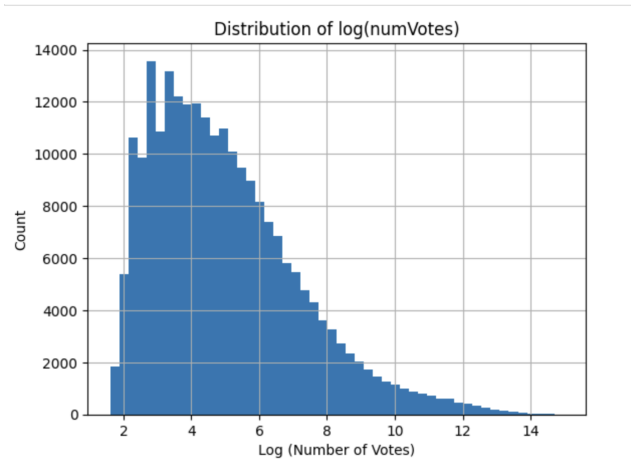


Figure 7: Distribution of Log Votes

The spread of the response variable - `averageRating` - seems approximately normal, though a bit left-skewed. I believe this to be due to the fact that rarely does a movie get a very low score. Many fans see a 4 or a 5 as an unacceptably low score, and often do not rate lower. The average seems to peak around 6.3, which makes sense for all movies from 1970 to 2025. There should not be issues with this in modeling, though there may be some predictions off more on the lower side than the higher side.

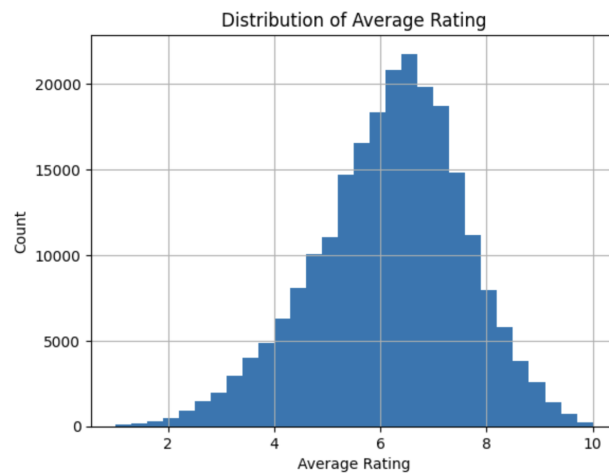


Figure 8: Distribution of Average Ratings

3 Methods

For each model, I will be using `averageRating` as the response, with `runtimeMinutes`, `startYear` and `genre` as the predictors. For linear/lasso/ridge the `Drama` indicator will be removed as it is needed as a baseline for interpretation, but it will be included for support vector regression and random forests.

3.1 Linear Regression

The choice for the baseline model is linear regression. Although there are more complex and tuned models, linear regression is still essential for interpretation, and a good baseline to compare to the regularized/nonlinear models.

I split the data with an 80/20 train/test split, and the same split will be used for the other models. I used a random seed of 748 for reproducible results.

Baseline linear regression uses ordinary least squares to minimize squared error, and $RMSE$ and R^2 are used as measures of model strength/predictive power.

The model returned an $RMSE = 1.233$ and $R^2 = 0.227$. This $RMSE$ is not awful, and is about within one point of rating (on a scale of 1-10), so is about 10% off. Other models may improve upon the $RMSE$ and can definitely improve the explained variance. The R^2 value is quite low, the noise within the data being the most likely culprit. There are many genre markers, and even though `startYear` was limited, it does increase a lot around the year 2000.

Figure 9 shows the positive coefficients and their strength, and figure 10 shows the same for the negative coefficients. It is evident that documentary has the strongest positive effect on rating while Horror has the strongest negative effect on rating. These both make sense from the idea that documentaries are often seen as a more neutral and academic genre while horror is often viewed as a lesser genre, and inspires push back from certain communities. `runtimeMinutes` is also a positive predictor, though is much lower than any genre, and `genre` does seem to be the strongest driver in `averageRating`.

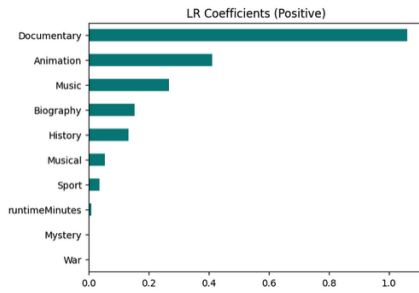


Figure 9: LR Coefficients (Positive)

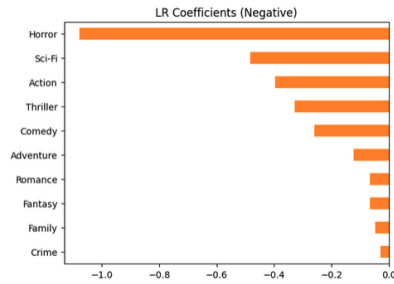


Figure 10: LR Coefficients (Negative)

3.1.1 Including logVotes

After modeling linear regression without `logVotes`, I decided to add it back into the model to see if it gave a significant difference in accuracy, predictive power, or explanation of variance. Running the model with this included gave identical $RMSE = 1.233$ and $R^2 = 0.227$ meaning that its inclusion in the model does not help predictive power and does not add to the model much. Its coefficient is -0.0107, but due to the small sign, its negative quality should not be considered in bringing down ratings.

3.2 Ridge and Lasso

The next model I chose was ridge and lasso. The motivation behind choosing this is to take advantage of the shrinkage and stability that ridge allows, and to use the shrinkage and variable selection provided by the lasso.

The same 80/20 train/test split was used, however I used the `StandardScaler()` function in python to prepare the data for modeling as they require scaled variables for their respective algorithms.

The first model ran is ridge regression, which uses an L_2 penalty and can be thought of as a modified OLS where there is a penalty term to balance complexity. With ridge we are seeking,

$$\min_{\beta} \text{RSS} + \lambda \sum \beta_j^2$$

This can help with stability and multicollinearity, though it will not actually perform any feature selection.

We find the best lambda value by testing values over a log-spaced grid using 5-fold cross-validation and use $RMSE$ for performance measures. With the ridge model we obtain $RMSE = 1.234$ which is slightly less than the linear regression model, though is similar enough to not show any significant difference in performance.

The second model ran is lasso regression. This uses an L_1 penalty and can double as feature selection to help develop a sparser model while also increasing our performance. Here we are minimizing,

$$\min_{\beta} \text{RSS} + \lambda \sum |\beta_j|$$

Similar to ridge, we find the best lambda value in a similar manner, though will be finding two this time. The first is for λ_{min} , which uses the lowest average MSE, and the second is λ_{1se} , which finds the maximum λ while keeping the error within 1 standard error of the minimum error. The former often keeps a large amount of coefficients while the latter is able to shrink the features even more. From python we obtain values of

$$\lambda_{min} = 0.001, \lambda_{1se} = 0.029$$

Due to the nature of the restrictions, it makes sense that our λ_{1se} is larger, since the goal is to maximize with a larger error bound.

Fitting both, we can measure performance using $RMSE$. and also see how many coefficients are dropped. For λ_{min} we get $RMSE = 1.234$ with 19 features retained, and for λ_{1se} we get $RMSE = 1.238$ with 11 features. These are both very similar to linear regression, and from figure 11 below, we can see that the same features are powerful predictors. An important note, this plot uses the absolute value to determine power of prediction, and does not include the sign. It is evident that Documentary and Horror seem to be the most important predictors, with runtimeMinutes following close behind.

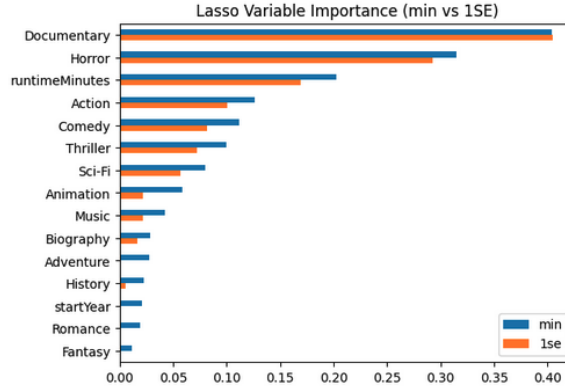


Figure 11: Lasso Variable Importance Comparison

The regularized versions of the OLS models do help with interpretation, especially in the case of lasso with λ_{1se} , however there is a bit of a loss of accuracy, and bias introduced via the regularization. The λ_{1se} model trades minimal accuracy for a simpler model. The $RMSE$ is comparable to linear regression, and the simplicity of that model would be preferred.

3.2.1 Including logVotes

After modeling lasso without $\logVotes$, I decided to add it back into the model to see if it gave a significant difference in accuracy, predictive power, or explanation of variance. Running the model with this included gave λ_{min} an $RMSE = 1.233$ and λ_{1se} and $RMSE = 1.238$ meaning that its inclusion in the model does not help predictive power and does not add much to the model.

3.3 Support Vector Regression

The next model is support vector regression (SVR). SVR uses a margin that will ignore small errors, while penalizing only large errors. with SVR we are seeking,

$$\min_{w,b} \frac{1}{2} \|w\|^2 + C \sum \max(0, |y_i - (w^T x_i + b)| - \epsilon)$$

I performed three versions of this, the first uses the linear kernel model, which uses the default parameters $C = 1, \epsilon = 0.1$. The second model uses the Radial Basis Function/Gaussian (RBF) kernel, untuned, which uses,

$$K(x_i, x) = \exp(-\gamma||x_i - x||^2)$$

Plugging into and optimizing the dual form, we obtain the prediction function

$$f(x) = \sum_{i=1}^n (\alpha_i - \alpha_i^*) \exp(-\gamma||x_i - x||^2) + b$$

This untuned model uses the equation above, and $C = 1, \epsilon = 0.1, \gamma = \text{auto}$. Here nonlinearity is introduced, but it is not tuned.

The third model again uses the RBF kernel, but uses `GridSearchCV()` to optimize the hyperparameters using cross validation (3-fold, for processing time).

I use a similar 80/20 train/test split here, however *Drama* is included as the interpretation is not as important in this model. I used the same seed of 748 to ensure the same split was created. For linear SVR the whole model was fit, however for RBF (both untuned and tuned), the data was again resampled to include only 10,000 rows as the algorithm is computationally expensive and took about 1.5 minutes to run. This means that it is not fully comparable to the full-data model, but it is close. I tested 2,000, 5,000, 10,000 and 50,000 and found that 10,000 optimized performance time and model performance.

For linear SVR I obtained $RMSE = 1.229$, for untuned RBF SVR I obtained $RMSE = 1.222$, and with the tuned RBF SVR I obtained $RMSE = 1.217$. These all perform better than linear regression, but like lasso and ridge, they are small improvements while introducing more complex structures that are unable to interpret.

3.4 Random Forests

The final model used is random forests. The benefit of the random forest is its ability to capture nonlinear interactions automatically. The random forests comes from the idea of a decision tree where we partition the space into regions. How random forests differ, is it generates many trees and averages the predictions over them (the forest). The random forests use bootstrap sampling to reduce correlation between trees.

I used two different random forests in this project. I first was untuned and used `n_estimators=200`, with the same random seed of 748. This returned $RMSE = 1.304$, which is the highest so far. The untuned random forest also gave the variable importance plot as seen in table 2 below. Of interesting note is that `runtimeMinutes` and `startYear` trump the genres that otherwise topped each other models features. Since this uses default complexity the next step was to tune the trees.

Variable	Importance
runtimeMinutes	0.2699
startYear	0.2298
Documentary	0.1748
Horror	0.0814
Drama	0.0334
Comedy	0.0225
Crime	0.0196
Romance	0.0178
Thriller	0.0174
Action	0.0166
Adventure	0.0161
Fantasy	0.0152
Mystery	0.0132
Family	0.0123
Sci-Fi	0.0095

Table 2: Untuned Random Forest Feature Importance

In this second model, I limited `max_depth` to 15, and `min_sample_leaf` to 5. This gave the best performance of any model so far with an $RMSE = 1.200$. This shows that limiting the depth of the forests allowed for a reduction in overfitting. This model also has the variable importance plot which aligns to the important features seen in each other model.

Variable	Importance
Documentary	0.3849
Horror	0.1792
runtimeMinutes	0.1581
startYear	0.0921
Drama	0.0688
Action	0.0206
Comedy	0.0149
Thriller	0.0125
Animation	0.0106
Sci-Fi	0.0106
Music	0.0091
Crime	0.0062
Biography	0.0054
Romance	0.0049
Mystery	0.0045

Table 3: Tuned Random Forest Feature Importance

4 Results

4.1 Model Performance Comparison

Model	RMSE
Linear Regression	1.233
Ridge (CV)	1.234
Lasso ($\lambda_{\min}$)	1.234
Lasso (λ_{1se})	1.238
Linear SVR	1.229
RBF SVR (untuned)	1.222
RBF SVR (tuned)	1.217
Random Forest (untuned)	1.304
Random Forest (tuned)	1.200

Table 4: Test RMSE across all fitted models

Table 4 above summarizes the *RMSE* for each model. It can be observed that most models have an *RMSE* around 1.2/1.3, and that the main difference in models is from an interpretation standpoint. An important note is also that the two RBF SVR models were trained on a slightly smaller dataset, though they gave similar performance.

When compared to the baseline linear regression, lasso and ridge do not improve the *RMSE* in a meaningful way. In fact, they all increased the *RMSE* a slight amount. What is good about these models, however, is that the important predictors are clear. The models clearly favor **Documentary**, **Horror**, and **runtimeMinutes** for predictions. This is true for each linear model.

The nonlinear SVR models improve the *RMSE* slightly, but not enough to consider them useful as their complex structure sacrifices interpretability. The random forests have both the best and worst performance. As they include a variable importance plot, these are solid for interpreting the features relationship to the **averageRating**.

4.2 Variable Importance

It is clear that for each model, besides the untuned random forest, that **Documentary**, **Horror**, and **runtimeMinutes** seem to be the most important features in predicting **averageRating**. If I want to go deeper, **startYear** is also an important predictor, makes some sense when considering that the user base on IMDb is of a specific age range, and individuals are often biased towards movies that were released during a specific portion of their life. Since most models agree with a small set of predictors, the underlying structure is stable.

5 Conclusion, Future Work

When comparing models, tuned random forests performed the best when measured by RMSE. The models also consistently show that **Documentary**, **Horror** and **runtimeMinutes** are the highest predictors of **averageRating**. It must be stated that IMDb has a specific user base, and that these conclusions do not generalize to America as a whole, and especially not the world.

Although satisfied with the conclusions reached, there were a few limitations imposed by the dataset. Firstly, there was no country predictor. This is an issue, since certain countries have varying average lengths of films as well as differing popular genres. A possible extension of the project could be to split up into categories like **Hollywood**, **Foreign**, **Independent**, etc. Secondly, **runtimeMinutes** was used as numeric variable, but movies generally fall into three categories: short, medium, and long. Were the project to be further explored, a formula would be developed to categorize movies into one of the three runtime categories. Finally, multiple data sets could be compared. There are numerous website that aggregate ratings and it would be interesting to compare the factors that impact scores.

6 AI Use

Chat GPT was used to help with python scripts and syntax. All words are my own.

7 Appendix: Code

```
#!/usr/bin/env python
# coding: utf-8

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.linear_model import LinearRegression, Ridge, RidgeCV, Lasso, LassoCV
from sklearn.metrics import mean_squared_error
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVR, LinearSVR
from sklearn.ensemble import RandomForestRegressor

pd.set_option("display.max_columns", None)
np.random.seed(748)

# Load data
basics = pd.read_csv(
    "data/title.basics.tsv.gz",
    sep='\t',
    compression='gzip',
    na_values="\N",
    low_memory=False
)

ratings = pd.read_csv(
    "data/title.ratings.tsv.gz",
    sep="\t",
    compression="gzip",
    na_values="\N"
)

# Filter movies
basic_info = basics[basics["titleType"] == "movie"].copy()
basic_info = basic_info[basic_info["isAdult"] == 0].copy()

basic_info = basic_info[["tconst", "primaryTitle", "startYear", "runtimeMinutes", "genres"]]
films = basic_info.merge(ratings, on="tconst", how="inner")

# Missingness analysis
films["runtime_missing"] = films["runtimeMinutes"].isna()
films["genre_missing"] = films["genres"].isna()

# Filter modern films
films_modern = films[films["startYear"] >= 1970]
films_clean = films_modern.dropna(subset=["runtimeMinutes", "genres"])

# One-hot encode genres
genres_coded = films_clean['genres'].str.get_dummies(',')
genre_means = genres_coded.mean()
keep_genres = genre_means[genre_means >= 0.01].index
genres_coded_filtered = genres_coded[keep_genres]

# Combine dataset
films_model = pd.concat([films_clean, genres_coded_filtered], axis=1)
films_model = films_model.drop(columns=["genres", "runtime_missing", "genre_missing"])

# Clean runtime
films_model["runtimeMinutes"] = pd.to_numeric(films_model["runtimeMinutes"], errors="coerce")
```

```

films_model = films_model[
    (films_model["runtimeMinutes"] >= 60) &
    (films_model["runtimeMinutes"] <= 300)
]

# Transform votes
films_model["logVotes"] = np.log(films_model["numVotes"])

# ----- LINEAR MODEL -----

X_linear = films_model.drop(
    columns=["tconst", "averageRating", "primaryTitle", "logVotes", "numVotes", "Drama"]
)
y_linear = films_model["averageRating"]

Xl_train, Xl_test, yl_train, yl_test = train_test_split(
    X_linear, y_linear, test_size=0.2, random_state=748
)

model = LinearRegression()
model.fit(Xl_train, yl_train)
y_pred = model.predict(Xl_test)

rmse = np.sqrt(mean_squared_error(yl_test, y_pred))
print("Linear RMSE:", rmse)

coef_lin = pd.Series(model.coef_, index=X_linear.columns)

coef_lin[coef_lin > 0].sort_values(ascending=False).head(10).plot(
    kind="barh", color="teal"
)
plt.title("LR Coefficients (Positive)")
plt.gca().invert_yaxis()
plt.show()

coef_lin[coef_lin < 0].sort_values().head(10).plot(
    kind="barh", color="darkorange"
)
plt.title("LR Coefficients (Negative)")
plt.show()

# ----- RIDGE / LASSO -----

scaler = StandardScaler()
Xl_train_scaled = scaler.fit_transform(Xl_train)
Xl_test_scaled = scaler.transform(Xl_test)

alphas = np.logspace(-3, 3, 50)

ridge_cv = RidgeCV(alphas=alphas, cv=5)
ridge_cv.fit(Xl_train_scaled, yl_train)
ridge = Ridge(alpha=ridge_cv.alpha_)
ridge.fit(Xl_train_scaled, yl_train)

y_pred_ridge = ridge.predict(Xl_test_scaled)
rmse_ridge = np.sqrt(mean_squared_error(yl_test, y_pred_ridge))
print("Ridge RMSE:", rmse_ridge)

lasso_cv = LassoCV(alphas=alphas, cv=5)
lasso_cv.fit(Xl_train_scaled, yl_train)

```

```

mse_mean = lasso_cv.mse_path_.mean(axis=1)
mse_std = lasso_cv.mse_path_.std(axis=1)

min_idx = np.argmin(mse_mean)
min_error = mse_mean[min_idx]

threshold = min_error + mse_std[min_idx]
alpha_1se = lasso_cv.alphas_[mse_mean <= threshold].max()

lasso_min = Lasso(alpha=lasso_cv.alpha_)
lasso_1se = Lasso(alpha=alpha_1se)

lasso_min.fit(Xl_train_scaled, yl_train)
lasso_1se.fit(Xl_train_scaled, yl_train)

y_pred_min = lasso_min.predict(Xl_test_scaled)
y_pred_1se = lasso_1se.predict(Xl_test_scaled)

print("Lasso RMSE:", np.sqrt(mean_squared_error(yl_test, y_pred_min)))
print("Lasso 1SE RMSE:", np.sqrt(mean_squared_error(yl_test, y_pred_1se)))

# ----- SVR -----

X_full = films_model.drop(
    columns=["tconst", "averageRating", "primaryTitle", "logVotes", "numVotes"]
)
y_full = films_model["averageRating"]

Xf_train, Xf_test, yf_train, yf_test = train_test_split(
    X_full, y_full, test_size=0.2, random_state=748
)

scaler = StandardScaler()
Xf_train_scaled = scaler.fit_transform(Xf_train)
Xf_test_scaled = scaler.transform(Xf_test)

svr_linear = LinearSVR(max_iter=10000)
svr_linear.fit(Xf_train_scaled, yf_train)

print("Linear SVR RMSE:",
      np.sqrt(mean_squared_error(yf_test, svr_linear.predict(Xf_test_scaled))))

# RBF (subset)
idx = np.random.choice(Xf_train_scaled.shape[0], size=10000, replace=False)
X_small = Xf_train_scaled[idx]
y_small = yf_train.iloc[idx]

svr_rbf = SVR(kernel='rbf')
svr_rbf.fit(X_small, y_small)

print("RBF SVR RMSE:",
      np.sqrt(mean_squared_error(yf_test, svr_rbf.predict(Xf_test_scaled))))

# Tuned RBF
param_grid = {
    "C": [0.1, 1, 10],
    "gamma": [0.01, 0.1, 1],
    "epsilon": [0.05, 0.1, 0.2]
}

```

```

grid = GridSearchCV(SVR(kernel="rbf"), param_grid, cv=3,
                    scoring="neg_mean_squared_error", n_jobs=-1)

grid.fit(X_small, y_small)

best_svr = grid.best_estimator_
print("Tuned SVR RMSE:",
      np.sqrt(mean_squared_error(yf_test, best_svr.predict(Xf_test_scaled))))

# ----- RANDOM FOREST -----

rf = RandomForestRegressor(n_estimators=200, random_state=748, n_jobs=-1)
rf.fit(Xf_train, yf_train)

print("RF RMSE:",
      np.sqrt(mean_squared_error(yf_test, rf.predict(Xf_test))))

rf_tune = RandomForestRegressor(
    n_estimators=200,
    max_depth=15,
    min_samples_leaf=5,
    random_state=748,
    n_jobs=-1
)

rf_tune.fit(Xf_train, yf_train)

print("Tuned RF RMSE:",
      np.sqrt(mean_squared_error(yf_test, rf_tune.predict(Xf_test))))

```